

read_point_clouds

July 16, 2026

```
[ ]: # Load libraries
import pye57
from pathlib import Path
from pathlib import Path
import numpy as np
import open3d as o3d
import plotly.graph_objects as go
import random

[ ]: # Initialize E57 reader
try:
    e57_path = "data/test_cloud.e57"
    # Test if the underlying C++ extension initializes safely
    print("pye57 imported successfully!")
except Exception as e:
    print(f"Error: {e}")

[ ]: # Path resolution
repo_root = Path.cwd().parent
e57_path = str(repo_root / "data" / "test_cloud.e57")

# Read file
e57 = pye57.E57(e57_path)

[ ]: # Read scan 0 with strict missing-field tolerances
## Uses parameter suggested by previous ValueError
data = e57.read_scan(0, intensity=True, colors=True, ignore_missing_fields=True)

# Check which fields successfully loaded
print("Successfully extracted fields from your E57:")
for key in data.keys():
    print(f" - {key}")

[ ]: # Extract raw coordinates
x = data["cartesianX"]
y = data["cartesianY"]
z = data["cartesianZ"]
```

```

# Combine into an (N, 3) matrix
points_3d = np.column_stack((x, y, z))
print(f"Spatial matrix shape: {points_3d.shape}")
print(f"Total points ready for processing: {points_3d.shape[0]:,}")

# Grab the scanner's transformation matrix
scanner_pose = e57.scan_position(0)
print("\nScanner Position Matrix (Pose):")
print(scanner_pose)

```

```

[ ]: # Initialize the Open3D PointCloud object
pcd = o3d.geometry.PointCloud()

# Load NumPy points into Open3D format
pcd.points = o3d.utility.Vector3dVector(points_3d)

# Voxel downsample to 5cm chunks
pcd_down = pcd.voxel_down_sample(voxel_size=0.05)

print(f"Original point cloud size: {len(pcd.points):,}")
print(f"Downsampled point cloud size: {len(pcd_down.points):,}")

```

```

[ ]: # Convert the Open3D downsampled cloud back to a NumPy array
down_points = np.asarray(pcd_down.points)

# Slice a subset of points for plotting
max_display_points = 100000
if len(down_points) > max_display_points:
    indices = np.random.choice(len(down_points), max_display_points,
                               replace=False)
    display_points = down_points[indices]
else:
    display_points = down_points

# Create 3D Scatter Plot
fig = go.Figure(data=[go.Scatter3d(
    x=display_points[:, 0],
    y=display_points[:, 1],
    z=display_points[:, 2],
    mode='markers',
    marker=dict(
        size=1.5,
        color=display_points[:, 2], # Color by height (Z-axis)
        colorscale='Viridis',
        opacity=0.8
    )
)])

```

```

# Set up layout
fig.update_layout(
    margin=dict(l=0, r=0, b=0, t=0),
    scene=dict(
        xaxis=dict(title='X'),
        yaxis=dict(title='Y'),
        zaxis=dict(title='Z'),
        aspectmode='data' # Keeps the 1:1:1 geometric proportions intact
    )
)

# Display
fig.show()

```

```

[ ]: # Reset active point cloud copy
active_pcd = o3d.geometry.PointCloud(pcd_down)

VOXEL_SIZE = 0.05
AREA_PER_POINT = VOXEL_SIZE * VOXEL_SIZE
MIN_PLANE_AREA_M2 = .6
MIN_POINTS_THRESHOLD = int(MIN_PLANE_AREA_M2 / AREA_PER_POINT)

all_plane_equations = []
all_plane_clouds = []

print("Starting targeted plane extraction...")
print("-> Allowing initial massive floor/ceiling planes.")
print("-> Blocking subsequent horizontal 'furniture slices' to force wall_
↳detection.\n")

iteration = 0
consecutive_skips = 0

while True:
    # Terminate if the cloud gets too small or if we are stuck skipping phantoms
    if len(active_pcd.points) < MIN_POINTS_THRESHOLD or consecutive_skips > 20:
        print("\nLoop Terminated: Reached point threshold or ran out of valid_
↳structures.")
        break

    # Run RANSAC to find the largest remaining mathematical sheet
    plane_model, inliers = active_pcd.segment_plane(
        distance_threshold=0.05,
        ransac_n=3,
        num_iterations=5000
    )

```

```

A, B, C, D = plane_model
num_points = len(inliers)
estimated_area = num_points * AREA_PER_POINT

# Check orientation: |C| > 0.80 means it's a horizontal surface
is_horizontal = abs(C) > 0.7

# Reject minor horizontal surfaces (furniture/stratified slice)
if is_horizontal and estimated_area < 30.0:
    consecutive_skips += 1

    # Add microscopic jitter to break up the phantom plane
    pts = np.asarray(active_pcd.points)
    # Create a noise matrix matching the shape of points
    jitter = np.random.normal(0, 0.0001, size=pts.shape)
    active_pcd.points = o3d.utility.Vector3dVector(pts + jitter)

    continue

# Verify minimum structural size requirement
if estimated_area >= MIN_PLANE_AREA_M2:
    iteration += 1
    consecutive_skips = 0 # Reset skip counter

    all_pts_matrix = np.asarray(active_pcd.points)
    detected_plane_pts = all_pts_matrix[inliers]

    all_plane_equations.append(plane_model)
    all_plane_clouds.append(detected_plane_pts)

    orientation_label = "Horizontal" if is_horizontal else "Vertical Wall"
    print(f"Plane #{iteration} [{orientation_label}]: Area =
↪{estimated_area:.2f} m² ({num_points:,} pts) | Eq: {A:.2f}x + {B:.2f}y + {C:.
↪2f}z + {D:.2f} = 0")

    # Remove points from active pool to be checked
    active_pcd = active_pcd.select_by_index(inliers, invert=True)
else:
    print(f"Stopping: Next largest plane ({estimated_area:.2f} m²) is below
↪the size threshold.")
    break

print(f"\nExtraction complete! Saved {iteration} high-confidence structural
↪planes.")

```

```

[ ]: # Initialize Plotly figure canvas
fig = go.Figure()

def sample_array(arr, max_pts=25000):
    if len(arr) > max_pts:
        return arr[np.random.choice(len(arr), max_pts, replace=False)]
    return arr

# Iterate through all extracted planes and color-code by orientation
for idx, (plane_pts, plane_eq) in enumerate(zip(all_plane_clouds,
↳all_plane_equations)):
    # Downsample this individual plane slightly just for display speed
    disp_plane = sample_array(plane_pts, max_pts=5000)

    A, B, C, D = plane_eq
    is_horizontal = abs(C) > 0.80

    # Assign colors based on architectural type
    if is_horizontal:
        color_string = 'rgb(44, 122, 123)' # Deep Teal for Floors/Ceilings
        legend_group = 'Horizontal Slabs'
        trace_name = f'Slab #{idx+1}'
    else:
        color_string = 'rgb(229, 62, 62)' # Bright Crimson Red for Walls
        legend_group = 'Vertical Walls'
        trace_name = f'Wall #{idx+1}'

    # Add plane layer
    fig.add_trace(go.Scatter3d(
        x=disp_plane[:, 0], y=disp_plane[:, 1], z=disp_plane[:, 2],
        mode='markers',
        marker=dict(size=1.5, color=color_string, opacity=0.85),
        name=trace_name,
        legendgroup=legend_group
    ))

# Extract and add remaining unsegmented points (noise/clutter)
remaining_pts = np.asarray(active_pcd.points)

# Tuned display settings for the remaining clutter:
# Increased size to 2.0 and opacity to 0.5 to make furniture and people highly
↳visible
disp_remaining = sample_array(remaining_pts, max_pts=35000)

if len(disp_remaining) > 0:
    fig.add_trace(go.Scatter3d(
        x=disp_remaining[:, 0], y=disp_remaining[:, 1], z=disp_remaining[:, 2],

```

```

        mode='markers',
        marker=dict(size=1, color='rgb(160, 174, 192)', opacity=0.5), # Solid
        ↪Medium Grey
        name='Unsegmented Clutter/Furniture'
    ))

# 4. Enforce structural integrity properties
fig.update_layout(
    margin=dict(l=0, r=0, b=0, t=0),
    scene=dict(
        aspectmode='data',
        xaxis=dict(backgroundcolor="rgb(247, 250, 252)", gridcolor="white"),
        yaxis=dict(backgroundcolor="rgb(247, 250, 252)", gridcolor="white"),
        zaxis=dict(backgroundcolor="rgb(247, 250, 252)", gridcolor="white")
    ),
    showlegend=True
)

fig.show()

```

```

[ ]: # 1. Initialize Plotly figure canvas
fig = go.Figure()

# 2. Slice down any left-over unsegmented room structure so it doesn't lag the
    ↪browser
all_pts_matrix = np.asarray(pcd_down.points)
max_display_pts = 30000

def sample_array(arr, max_pts=25000):
    if len(arr) > max_pts:
        return arr[np.random.choice(len(arr), max_pts, replace=False)]
    return arr

# 3. Add the discovered planes one by one to the visualizer
# We will iterate through all_plane_clouds using an index
for idx, plane_pts in enumerate(all_plane_clouds):
    # Downsample this individual plane slightly just for display speed
    disp_plane = sample_array(plane_pts, max_pts=5000)

    # Generate a random bright color for this specific plane layer
    r = random.randint(50, 255)
    g = random.randint(50, 255)
    b = random.randint(50, 255)
    color_string = f'rgb({r},{g},{b})'

    # Add this plane as its own individual "trace" layer
    fig.add_trace(go.Scatter3d(

```

```

        x=disp_plane[:, 0], y=disp_plane[:, 1], z=disp_plane[:, 2],
        mode='markers',
        marker=dict(size=1.5, color=color_string, opacity=0.8),
        name=f'Plane #{idx+1}'
    ))

# 4. Extract and add whatever was left unsegmented in the active cloud (the
↳noise/clutter)
remaining_pts = np.asarray(active_pcd.points)
disp_remaining = sample_array(remaining_pts, max_pts=20000)

if len(disp_remaining) > 0:
    fig.add_trace(go.Scatter3d(
        x=disp_remaining[:, 0], y=disp_remaining[:, 1], z=disp_remaining[:, 2],
        mode='markers',
        marker=dict(size=1, color='lightgrey', opacity=0.3),
        name='Unsegmented Clutter'
    ))

# 5. Maintain the true architectural scale (1:1:1)
fig.update_layout(
    margin=dict(l=0, r=0, b=0, t=0),
    scene=dict(aspectmode='data'),
    showlegend=True # This adds a clickable legend to toggle individual planes
↳on/off!
)

fig.show()

```

```

[ ]: ### Note to self: Currently searching for the lowest major flat slab to set
↳floor height level. Could do that wall by wall but run into issues with not
↳capturing data all the way down to the floor when walls are obscured.

```

```

# 1. Establish floor baseline
horizontal_slab_heights = []
for eq in all_plane_equations:
    A, B, C, D = eq
    if abs(C) > 0.7:
        horizontal_slab_heights.append(-D / C)
floor_z = min(horizontal_slab_heights) if len(horizontal_slab_heights) > 0 else
↳0.0

GAP_MAX_EMPTY_SPACE_M = 0.80
DOOR_HEIGHT_THRESHOLD_M = 0.30

segmented_wall_rectangles = []
detected_doorways = []

```

```

for wall_idx, wall_pts in enumerate(all_plane_clouds):
    A, B, C, D = all_plane_equations[wall_idx]
    if abs(C) > 0.7 or len(wall_pts) < 50:
        continue

    # Run PCA on the wall's horizontal footprint
    xy_pts = wall_pts[:, :2]
    xy_mean = np.mean(xy_pts, axis=0)
    centered_xy = xy_pts - xy_mean
    covariance_matrix = np.cov(centered_xy.T)

    eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix)
    wall_dir = eigenvectors[:, np.argmax(eigenvalues)]

    # Project to local 2D coordinates (Length, Height)
    lengths = centered_xy @ wall_dir
    heights = wall_pts[:, 2]

    # Global height ceiling for this specific wall to keep the top edge flat
    global_wall_max_z = heights.max()

    # Sort left-to-right along the wall's true face
    sort_idx = np.argsort(lengths)
    sorted_lengths = lengths[sort_idx]
    sorted_3d_pts = wall_pts[sort_idx]

    # Identify gaps
    gaps = np.diff(sorted_lengths)
    split_indices = np.where(gaps > GAP_MAX_EMPTY_SPACE_M)[0]

    start_idx = 0
    wall_segments = []
    for split in split_indices:
        wall_segments.append((start_idx, split))
        start_idx = split + 1
    wall_segments.append((start_idx, len(sorted_lengths) - 1))

    # Process each continuous segment
    for i in range(len(wall_segments)):
        seg_start, seg_end = wall_segments[i]
        if (seg_end - seg_start) < 10:
            continue

        min_len = sorted_lengths[seg_start]
        max_len = sorted_lengths[seg_end]

```

```

        # FIX: Force vertical limits to align perfectly with the floor and a
        ↪ flat ceiling
        min_z = floor_z
        max_z = global_wall_max_z

        # Un-project horizontal endpoints back into angled global 3D space
        pt_start_xy = xy_mean + min_len * wall_dir
        pt_end_xy = xy_mean + max_len * wall_dir

        corners_3d = np.array([
            [pt_start_xy[0], pt_start_xy[1], min_z], # 0: Bottom Left
            [pt_end_xy[0], pt_end_xy[1], min_z], # 1: Bottom Right
            [pt_end_xy[0], pt_end_xy[1], max_z], # 2: Top Right
            [pt_start_xy[0], pt_start_xy[1], max_z] # 3: Top Left
        ])

        segmented_wall_rectangles.append({
            'corners': corners_3d,
            'wall_idx': wall_idx
        })

        # Analyze the gap right after this segment for doors
        if i < len(wall_segments) - 1:
            next_seg_start = wall_segments[i+1][0]

            gap_center_x = (sorted_3d_pts[seg_end][0] +
        ↪ sorted_3d_pts[next_seg_start][0]) / 2.0
            gap_center_y = (sorted_3d_pts[seg_end][1] +
        ↪ sorted_3d_pts[next_seg_start][1]) / 2.0

            local_z_min = min(sorted_3d_pts[seg_end][2],
        ↪ sorted_3d_pts[next_seg_start][2])
            if abs(local_z_min - floor_z) < DOOR_HEIGHT_THRESHOLD_M:
                detected_doorways.append(np.array([gap_center_x, gap_center_y,
        ↪ floor_z + 1.0]))

print(f"Generated {len(segmented_wall_rectangles)} true angled wall panels.")

```

```

[ ]: # Visualization Code
fig = go.Figure()

def sample_array(arr, max_pts=25000):
    if len(arr) > max_pts:
        return arr[np.random.choice(len(arr), max_pts, replace=False)]
    return arr

# 1. Plot Raw Structure Data

```

```

added_wall_legend, added_slab_legend = False, False
for idx, (plane_pts, plane_eq) in enumerate(zip(all_plane_clouds,
↳all_plane_equations)):
    disp_plane = sample_array(plane_pts, max_pts=5000)
    is_horizontal = abs(plane_eq[2]) > 0.70

    color = 'rgb(44, 122, 123)' if is_horizontal else 'rgb(229, 62, 62)'
    group = 'Horizontal Slabs' if is_horizontal else 'Vertical Walls'
    show_lgd = not added_slab_legend if is_horizontal else not added_wall_legend
    if is_horizontal: added_slab_legend = True
    else: added_wall_legend = True

    fig.add_trace(go.Scatter3d(
        x=disp_plane[:, 0], y=disp_plane[:, 1], z=disp_plane[:, 2],
        mode='markers',
        marker=dict(size=1.5, color=color, opacity=0.3),
        name=group, legendgroup=group, showlegend=show_lgd
    ))

# 2. Plot Truly Aligned Solid Wall Overlays
added_overlay_legend = False
for box in segmented_wall_rectangles:
    c = box['corners']
    show_lgd = not added_overlay_legend
    added_overlay_legend = True

    fig.add_trace(go.Mesh3d(
        x=c[:, 0], y=c[:, 1], z=c[:, 2],
        i=[0, 0], j=[1, 2], k=[2, 3],
        color='rgb(246, 173, 85)',
        opacity=0.6,
        name='Completed Structural Panels',
        legendgroup='Structural Panels',
        showlegend=show_lgd
    ))

# 3. Plot Doors
if len(detected_doorways) > 0:
    doors_matrix = np.array(detected_doorways)
    fig.add_trace(go.Scatter3d(
        x=doors_matrix[:, 0], y=doors_matrix[:, 1], z=doors_matrix[:, 2],
        mode='markers',
        marker=dict(size=8.0, color='rgb(72, 187, 120)', symbol='diamond'),
        name='Isolated Doorways', legendgroup='Doorways'
    ))

# 4. Plot Furniture

```

```

remaining_pts = np.asarray(active_pcd.points)
disp_remaining = sample_array(remaining_pts, max_pts=35000)
if len(disp_remaining) > 0:
    fig.add_trace(go.Scatter3d(
        x=disp_remaining[:, 0], y=disp_remaining[:, 1], z=disp_remaining[:, 2],
        mode='markers',
        marker=dict(size=1.0, color='rgb(160, 174, 192)', opacity=0.4),
        name='Unclassified Furniture', legendgroup='Furniture'
    ))

fig.update_layout(
    margin=dict(l=0, r=0, b=0, t=0),
    scene=dict(aspectmode='data'),
    showlegend=True
)
fig.show()

```

```

[ ]: # Print every top-level key and node type in the E57 file structure
root_node = e57.image_file.root()

print("--- Root Level Nodes Available ---")
for i in range(len(root_node)):
    child = root_node[i]
    print(f"Node Name: {child.elementName()} | Type: {type(child).__name__}")
images_node = e57.image_file.root()["images2D"]
print(f"Found {len(images_node)} trajectory keyframes inside images2D.\n")

trajectory_points = []

for i in range(len(images_node)):
    img = images_node[i]

    try:
        # Explicitly pull the pose structure node using pye57's node lookup
        pose = img["pose"]
        trans = pose["translation"]

        # Pull coordinates directly by calling .value() or accessing float nodes
        tx = trans["x"].value()
        ty = trans["y"].value()
        tz = trans["z"].value()

        trajectory_points.append([tx, ty, tz])

    print(f"Waypoint #{i}: Scanner Position = [{tx:.3f}, {ty:.3f}, {tz:.3f}]\n")

```

```

except Exception as e:
    # Fallback: Check if the scanner stored positions directly in visual_
↳structures
    try:
        # Alternate standard E57 camera node naming structure
        pose = img["pose"]
        tx = pose["translation"]["x"].value()
        ty = pose["translation"]["y"].value()
        tz = pose["translation"]["z"].value()
        trajectory_points.append([tx, ty, tz])
        print(f"Waypoint #{i} (Fallback): Position = [{tx:.3f}, {ty:.3f},
↳{tz:.3f}"] )
    except:
        print(f"Could not parse Waypoint #{i} directly. Error signature:
↳{str(e)}")

trajectory_points = np.array(trajectory_points)
print(f"\nSuccessfully compiled trajectory matrix with shape:
↳{trajectory_points.shape}")

```

[]: